LAB # 01
Implementation of basic structure of java programming environment.

Objective

To get a hands on training exposure to the Java development environment.

Lab Goals

To demonstrate and gain familiarity with the various operations on Sun's Java development environment including:

a. Getting knowledge of jdk,jre and jvm.
b. Installing Netbeans 8.2 along with jdk on your PC.
c. Creating a Java Program
d. Saving the program
e. Compiling & debugging
f. Executing the program.

INTRODUCTION TO JAVA
Java is a high-level, object-oriented programming language developed by Sun Microsystems (now owned by Oracle). It is known for its platform independence, meaning Java programs can run on any device that has a Java Virtual Machine (JVM) installed.

INTRODUCTION TO BASIC JAVA ENVIRONMENT: To write and run Java programs, you need to set up a basic environment consisting of:

1. Java Development Kit (JDK)
2. Integrated Development Environment (IDE) or a text editor
3. Command-line tools (optional)

JAVA DEVELOPMENT KIT

The Java Development Kit (JDK) is a software development environment used for developing Java applications and applets. It includes the Java Runtime Environment (JRE), an interpreter/loader (Java), a compiler (javac), an archiver (jar), a documentation generator (Javadoc) and other tools needed in Java development.

JAVA RUNTIME ENVIRONMENT

JRE stands for **"Java Runtime Environment"** and may also be written as **"Java RTE."** The Java Runtime Environment provides the minimum requirements for executing a Java application; it consists of the *Java Virtual Machine (JVM), core classes*, and *supporting files*.

JAVA VIRTUAL MACHINE:

- A **specification** where working of Java Virtual Machine is specified. But implementation provider is independent to choose the algorithm. Its implementation has been provided by Sun and other companies.
- An **implementation** is a computer program that meets the requirements of the JVM specification
- **Runtime Instance** Whenever you write java command on the command prompt to run the java class, an instance of JVM is created.

Difference betweem JDK, JRE and JVM

To understand the difference between these threes

- **JDK** – **Java Development Kit** (in short JDK) is Kit which provides the environment to **develop and execute(run)** the Java program. JDK is a kit(or package) which includes two things
  1. Development Tools(to provide an environment to develop your java programs)
  2. JRE (to execute your java program).

  Note : JDK is only used by Java Developers.
- **JRE** – **Java Runtime Environment** (to say JRE) is an installation package which provides environment to **only run(not develop)** the java program(or application)onto your machine. JRE is only used by them who only wants to run the Java Programs i.e. end users of your system.
- **JVM** – **Java Virtual machine**(JVM) is a very important part of both JDK and JRE because it is contained or inbuilt in both. Whatever Java program you run using JRE or JDK goes into JVM and JVM is responsible for **executing the java program line by line** hence it is also known as interpreter.

What does JRE consists of?

JRE consists of the following components:

- **Deployment technologies**, including deployment, Java Web Start and Java Plug-in.
- **User interface toolkits**, including *Abstract Window Toolkit (AWT), Swing, Java 2D, Accessibility, Image I/O, Print Service, Sound, drag and drop (DnD)* and *input methods*.
- **Integration libraries**, including *Interface Definition Language (IDL), Java Database Connectivity (JDBC), Java Naming and Directory Interface (JNDI), Remote Method Invocation (RMI), Remote Method Invocation Over Internet Inter-Orb Protocol (RMI-IIOP)* and *scripting*.
- **Other base libraries**, including *international support, input/output (I/O), extension mechanism, Beans, Java Management Extensions (JMX), Java Native Interface (JNI), Math, Networking, Override Mechanism, Security, Serialization* and *Java for XML Processing (XML JAXP)*.
- **Lang and util base libraries**, including *lang and util, management, versioning, zip, instrument, reflection, Collections, Concurrency Utilities, Java Archive (JAR), Logging, Preferences API, Ref Objects* and *Regular Expressions.*
- **Java Virtual Machine (JVM)**, including *Java HotSpot Client* and *Server Virtual Machines.*

How does JRE works?

To understand how the JRE works let us consider a Java source file saved as *Example.java*. The file is compiled into a set of Byte Code that is stored in a "*.class*" file. Here it will be "*Example.class*".

The following actions occur at runtime.

- **Class Loader**

The Class Loader loads all necessary classes needed for the execution of a program. It provides security by separating the namespaces of the local file system from that imported through the network. These files are loaded either from a hard disk, a network or from other sources.

- **Byte Code Verifier**

The JVM puts the code through the Byte Code Verifier that checks the format and checks for an illegal code. Illegal code, for example, is code that violates access rights on objects or violates the implementation of pointers.

The Byte Code verifier ensures that the code adheres to the JVM specification and does not violate system integrity.

- **Intrepreter**

At runtime the Byte Code is loaded, checked and run by the interpreter. The interpreter has the following two functions:

- Execute the Byte Code
- Make appropriate calls to the underlying hardware

How does JVM works?

JVM becomes an instance of JRE at runtime of a Java program. It is widely known as a runtime interpreter.JVM largely helps in the abstraction of inner implementation from the programmers who make use of libraries for their programmes from JDK.

INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING (OOP): Object-oriented programming (OOP) is a programming paradigm based on the concept of "objects", which can contain data in the form of fields (often known as attributes or properties), and code in the form of procedures (often known as methods). Here are some key Object-Oriented Concepts in Java:

1. **Classes and Objects:** A class is a blueprint or template for creating objects. It defines the properties and behaviors (methods) that objects of the class will have. An ***object*** is an instance of a class. It represents a real-world entity and can be created using the new keyword followed by the class constructor.

2. **Inheritance:** Inheritance is a mechanism where a ***child class (subclass)*** is derived from an ***existing class/ parent class (superclass).*** The subclass inherits properties and behaviors (methods) of the superclass. It can also have its own additional properties and behaviors. In Java, inheritance is achieved using the ***extends*** keyword.

3. **Polymorphism:** Polymorphism allows objects of different classes to be treated as objects of a common superclass. There are two types of polymorphism in Java: ***compile-time polymorphism (method overloading) and runtime polymorphism (method overriding)***. Method overriding occurs when a subclass provides a specific implementation of a method that is already defined in its superclass.

4. **Abstraction:** Abstraction is the process of hiding the implementation details and showing only the essential features of the object. ***Abstract classes and interfaces*** are used to achieve abstraction in Java. Abstract classes cannot be instantiated and may contain abstract methods (methods without a body) that must be implemented by subclasses. Interfaces define a contract for classes to follow by specifying method signatures without implementations.

5. **Encapsulation:** Encapsulation is the bundling of data (attributes) and methods that operate on the data into a single unit (class). It helps in hiding the internal state of an object from the outside world and only exposing the necessary functionality. Access to the data is typically controlled using access modifiers ***(public, private, protected)***.

PROCEDURAL AND OBJECT-ORIENTED LANGUAGES: Procedural languages focus on procedures or routines to accomplish tasks, while object-oriented languages focus on objects and their interactions. Java is primarily an object-oriented language, but it also supports procedural programming.

Example using Procedural Approach

```
public class ProceduralExample {
    // Method to calculate the sum of two numbers
    public static int sum(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        int num1 = 5;
        int num2 = 10;

        // Call the sum method and print the result
        int result = sum(num1, num2);
        System.out.println("Sum of " + num1 + " and " + num2 + " is: " + result);
    }
}
```

In this example, we define a method ***sum()*** that calculates the sum of two integers. Then, in the ***main method***, we call this method with two numbers and print the result.

Example using Object-Oriented Approach

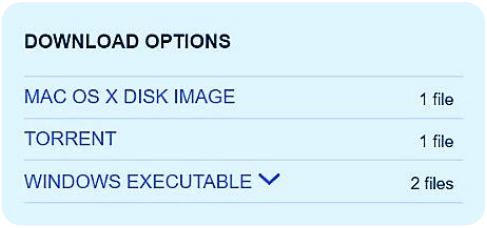
```
// Class representing a Rectangle
class Rectangle {
    private int width;
    private int height;

    // Constructor
    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    // Method to calculate area
    public int calculateArea() {
        return width * height;
    }
}
public class ObjectOrientedExample {
    public static void main(String[] args) {
        // Create an object of Rectangle class
        Rectangle rect = new Rectangle(5, 10);
```

```
        // Call the calculateArea method and print the result
        int area = rect.calculateArea();
        System.out.println("Area of rectangle is: " + area);
    }
}
```

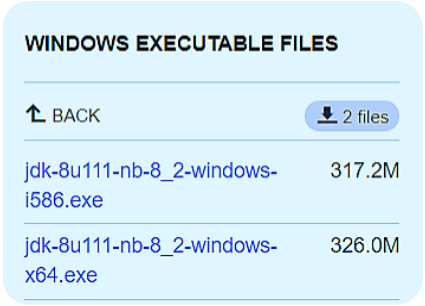
In this example, we define a class ***Rectangle*** with data members ***width*** and ***height***, a ***constructor*** to initialize these members, and a method ***calculateArea*** to compute the area of the rectangle. Then, in the ***main method***, we create an ***object*** of the Rectangle class and call its calculateArea method.

STEPS TO INSTALL JDK WITH NETBEANS 8.2 IDE: Follow these steps;

1. Visit to the link (https://archive.org/details/jdk-8u111-nb-8_2)
2. Select from Download Options

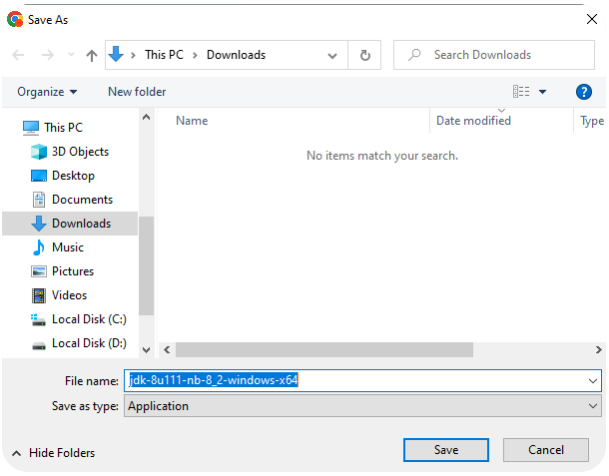


3. Select Windows Executable

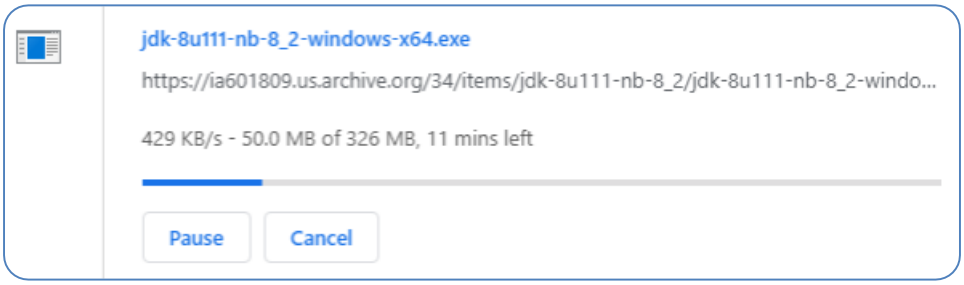


4. Choose a Download Option Windows operator 32 / 64 bit

5. Choose a download location on computer/laptop



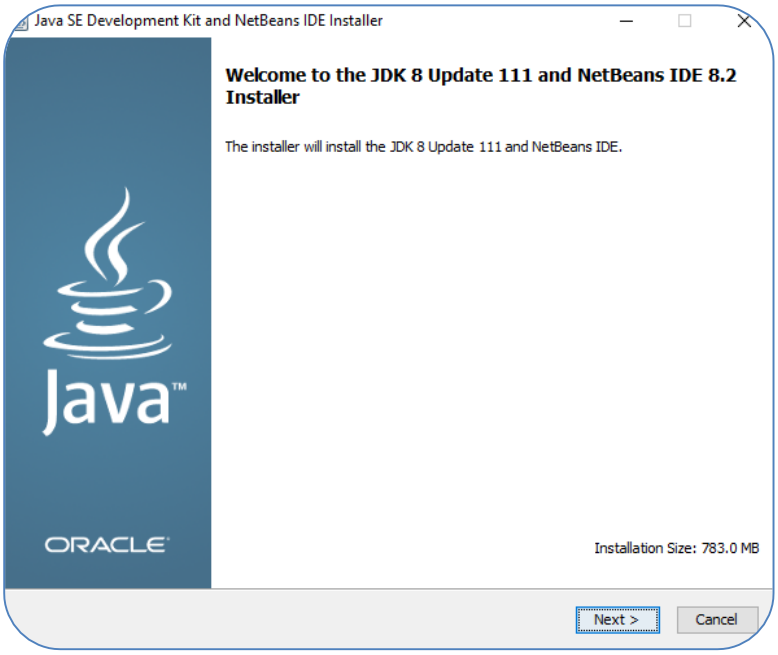
6. Download Progress



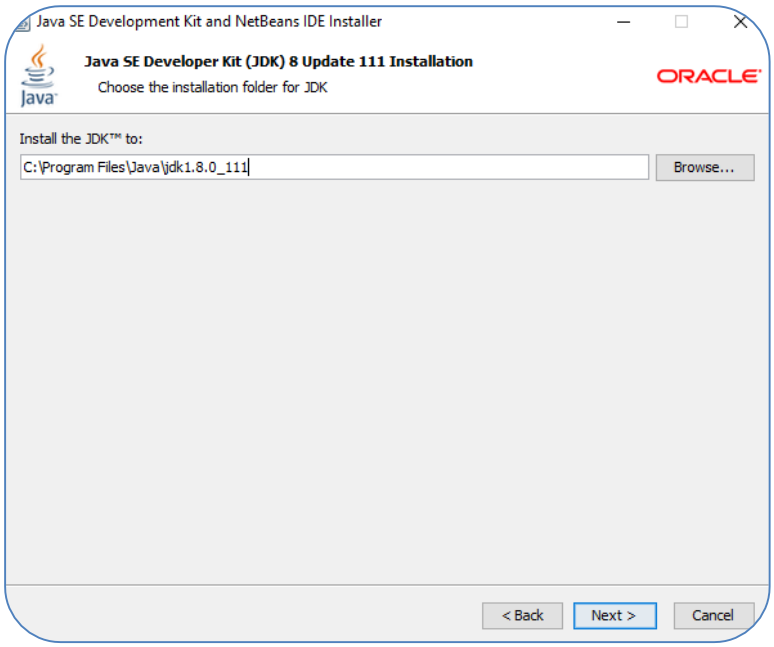
7. Download is successfully completed

8. Open The Downloaded Application

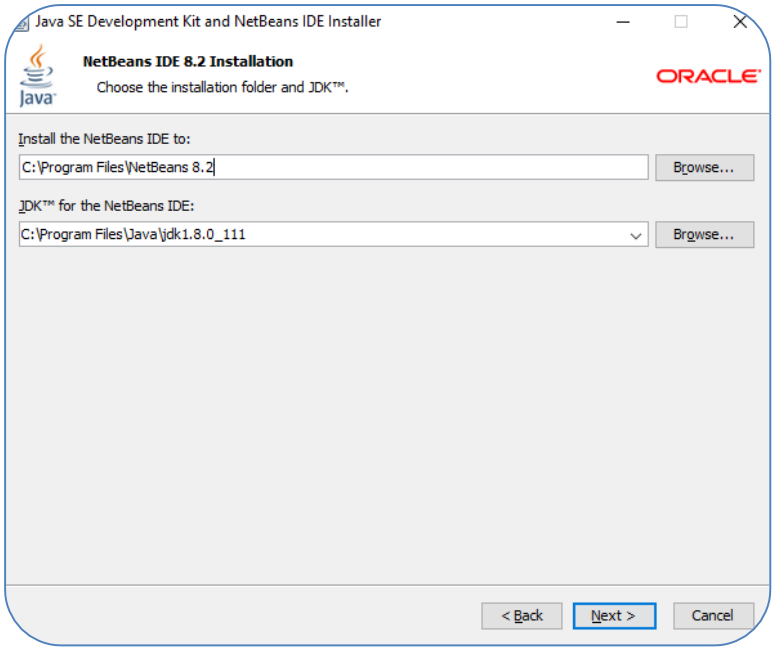
9. Click On Next



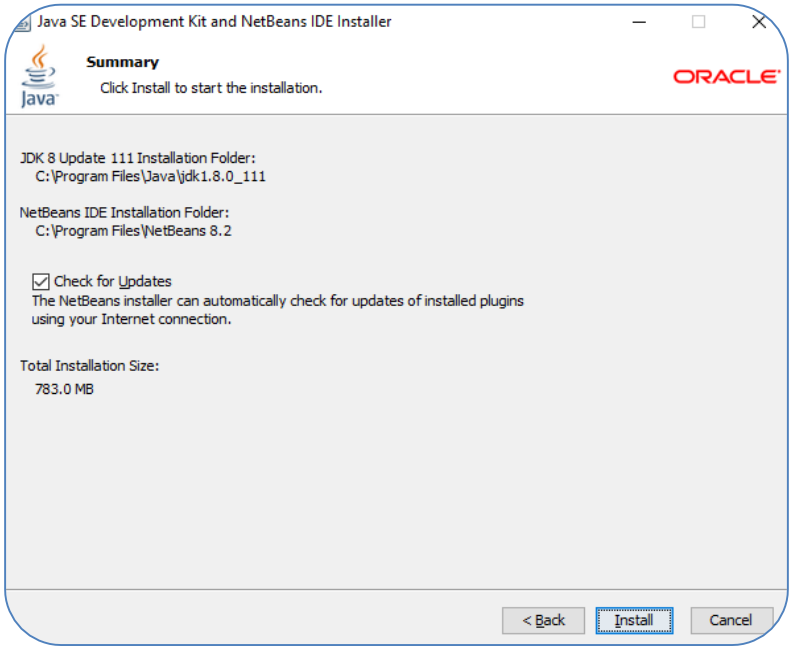
10. Install JDK in C:\Program Files\Java\jdk1.8.0_111, Click on Next



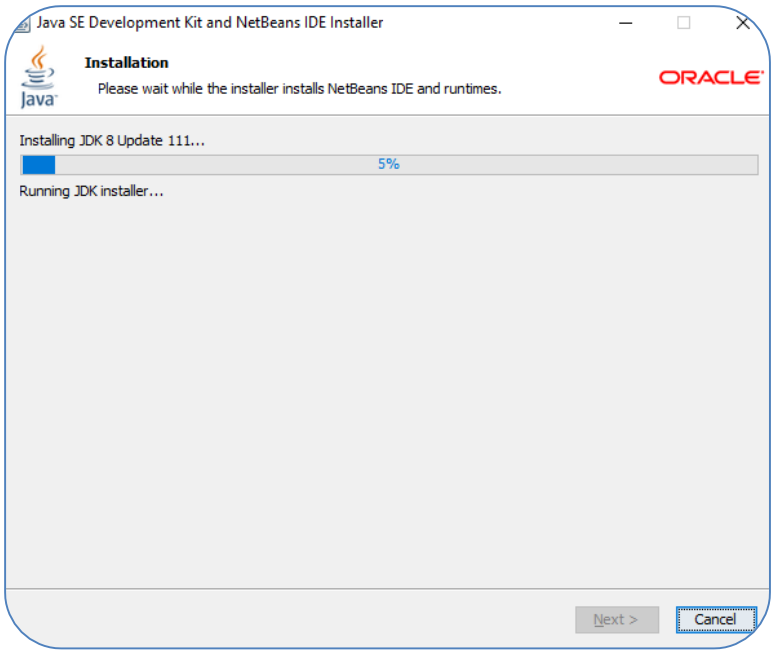
11. Install JDK & Net Beans IDE of JAVA, Click on Next



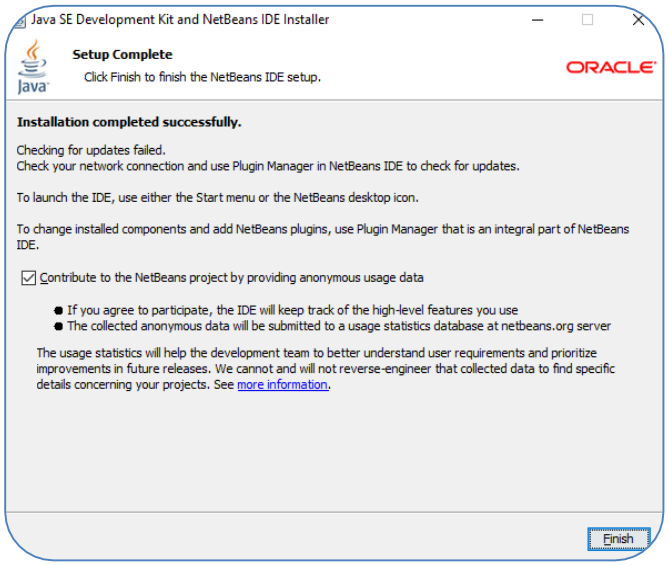
12. Click On Install



13. Installation in Progress



14. Click on Finish to complete the installation



Hence installation is successfully completed.

Caution about PATH & CLASSPATH

A common error with the beginners is defining the path and classpath statements. These statements are absolute necessary in order to create a better java work environment. We suggest that you create a separate folder on your root directory under name of "Javawork" and carry out all your practice code in that directory. Jdk3 by default looks for the java compiler and class libraries within its installation directory only. Therefore in order to make Java "see" your work directory, please add the following to your autoexec.bat file:

Path c:\javawork, c:\program files\jdk1.3\
Classpath c:\javawork, c:\program files\jdk1.3

Creating a Java Program

Java programs can be written and created using any standard text editor. JDK1.2 does not come with its native program editor as in Turbo C or other language compilers. Hence we can use anything-standard ASCII text editor to create the java programs. These may range from your regular run-of-the-mill MS-DOS blue screen editor invoked by "Edit <filename>. java command, to the notepad or WordPad accessories on GUI desktops and "ed" on Linux / Unix workstations

Important

No matter what editor you choose, always save the file with the extension ".java". Java compiler only looks to the program, which have the java extension.

Exercise: Now create the following Java program in your editor of choice and save it as "Isttask.java" in c:\javawork.

//program begins

```
public class {
   public static void main (String [] args)
  {
    System.out.println ("this is ist task");
  }
}
```

Important

In the exercise above, note that the class name is Isttask; same as the file name i.e. Bismillah.java. Infect it is a requirement of jdk2 that the class name of the file matches exactly word by word and case by case to the file name it is being created in.

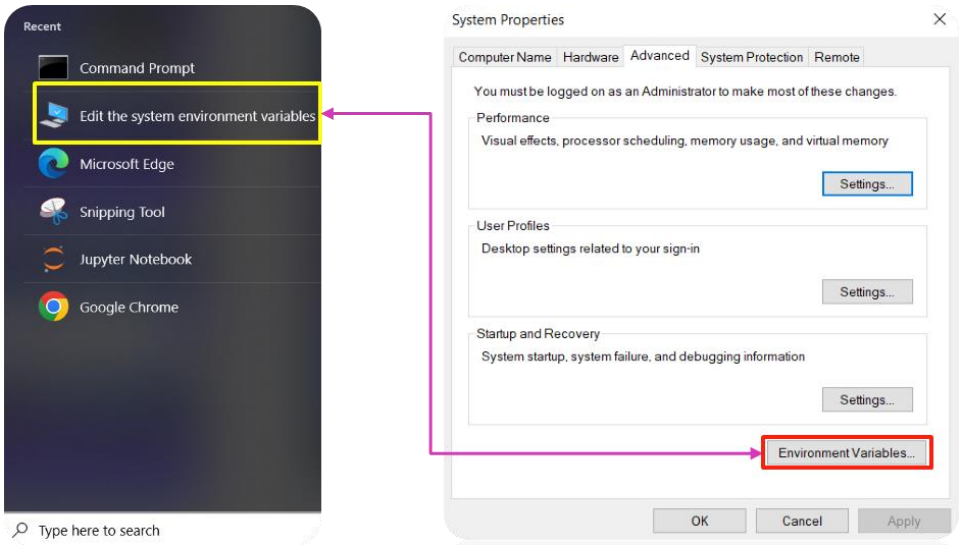
Java Folder: Organizing Java projects into folders is a good practice. A typical Java project structure might include folders for source code (src), compiled code (bin), libraries (lib), and resources (res).

Compilation and Running Process: Java programs are first compiled into bytecode using the Java compiler (javac). Then, the bytecode is executed by the Java Virtual Machine (java). The compilation process generates .class files, which are then executed using the java command.

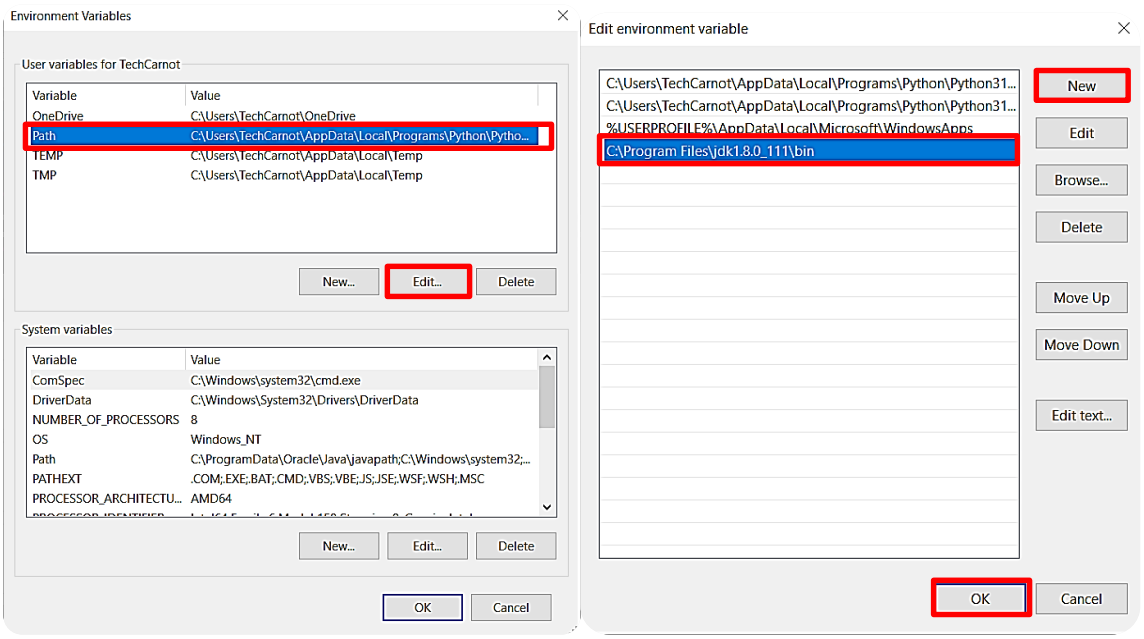
Note the "javac" in the line above. Here we are calling the java compiler (c stands for compiler). Here we are telling the compiler to compile or to be precise, generate byte code of the program Bismillah.java.
If the program is written as given above, it will be compiled without errors. If not, it will throw exceptions, which you need to fix before we proceed.

STEPS TO COMPILE AND RUN JAVA FILE: Follow

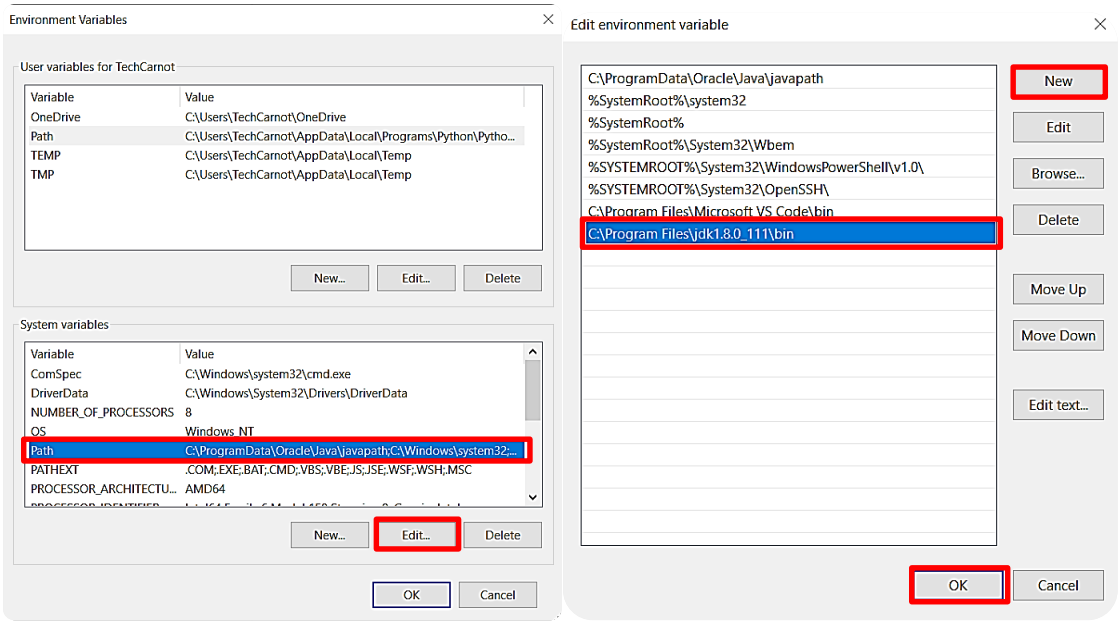
1. Open Edit system environment variables to set the path of java JDK.



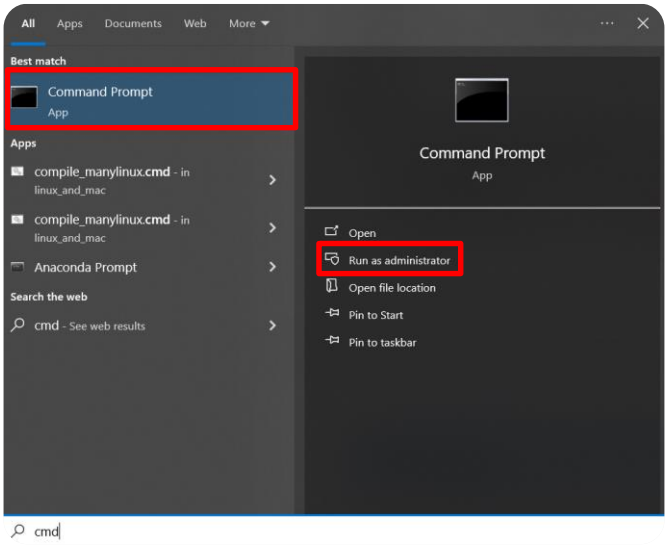
2. Set the path inside Environment Variable. (**C:\Program Files\Java\jdk1.8.0_111\bin**)



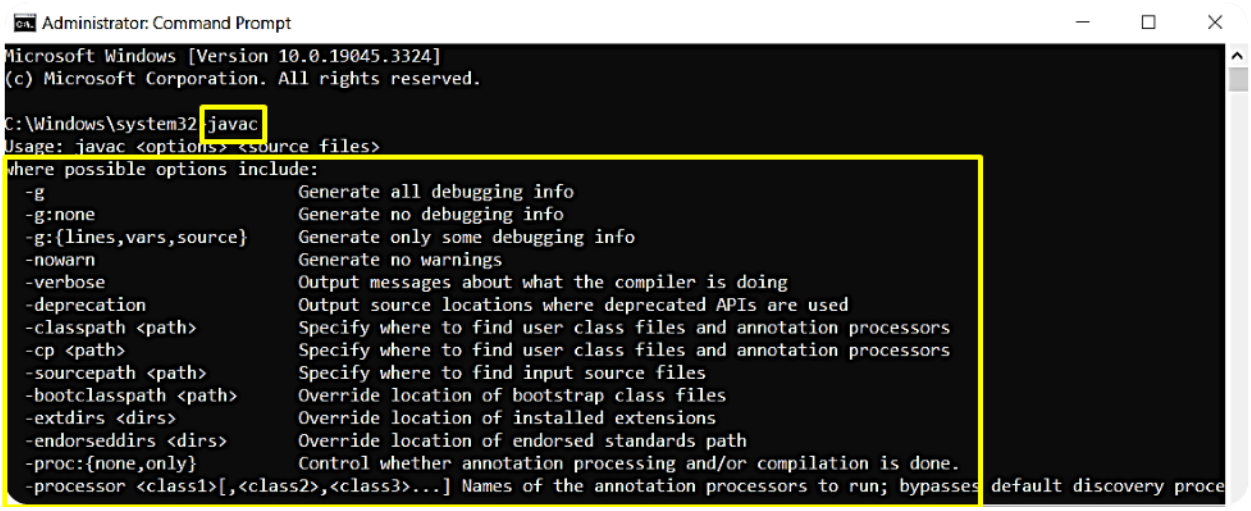
3. Set the path inside Environment Variable.



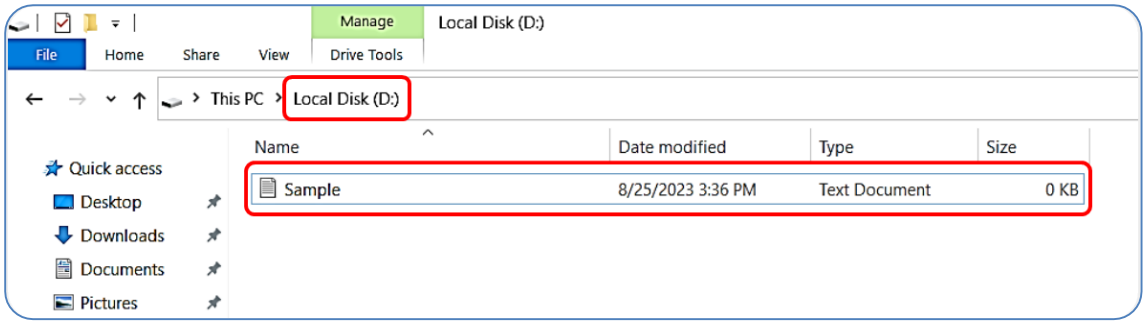
4. Open Command prompt & select run as an administrator.



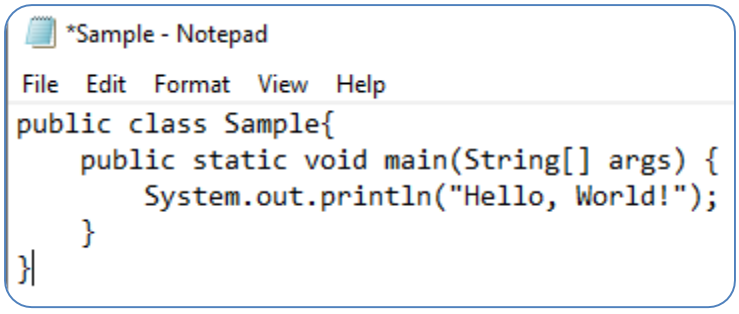
5. Inside Command prompt write javac to ensure that path is correctly set or not?



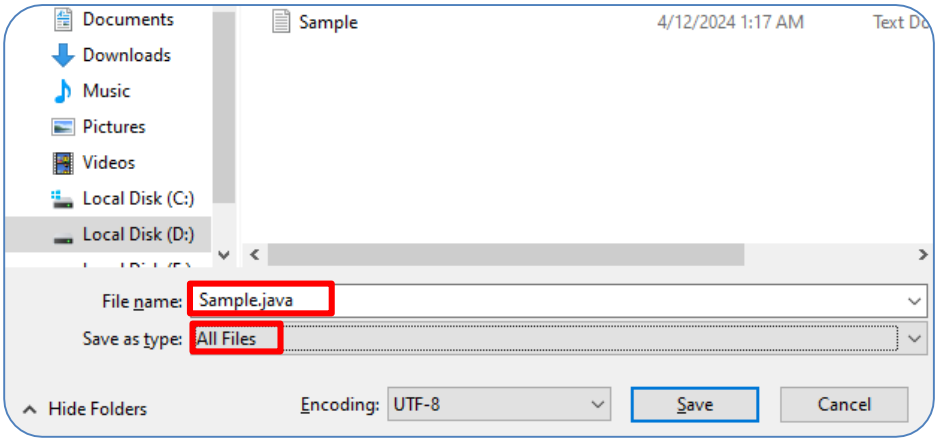
6. Then go to any directory suppose D:/ drive directory and create Sample file which is of **.txt format**.

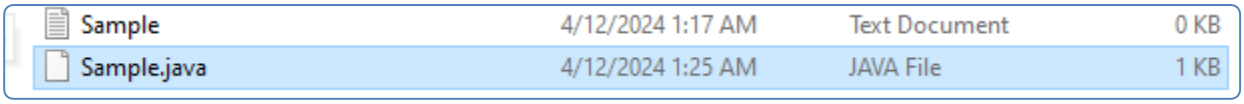


7. After that write the Java Code inside the Sample File.



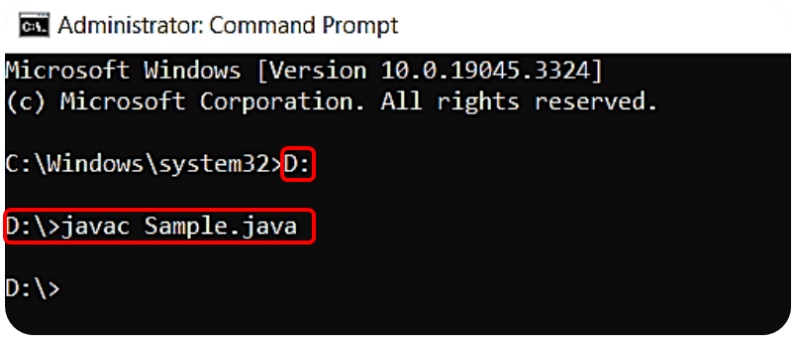
8. Now Save the file as **Sample.java** File which is of **Java File Format** & **close** the previous file.



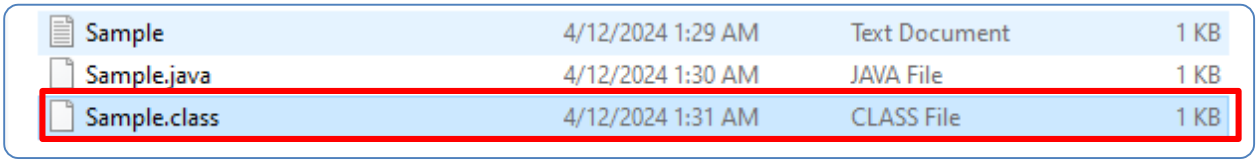


Now our Sample.java File which is of Java File Format has been created successfully.

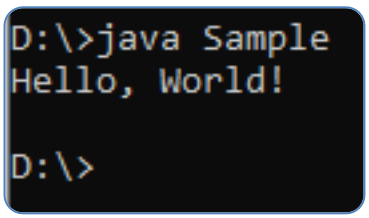
9. Write **D:/** in Command Prompt & after that write ***javac Sample.java*** in Command Prompt.



10. Now we see in the **D:/** drive directory that a **Sample.class** is generated.



11. Then write **java Sample** and after this program executed successfully & printing the result.



Exercise

A. Explore the editors used for java programming.
B. Write a program to take several attributes (i.e. name, age, gender, department, email id, father name) and print the result along with the greeting message.
C. Write a program to print your name, age, gender, department, email id, father name and print the result using Notepad. (Hint: Compile and Run Java File using Notepad.)